

IX

09/09

Validations & Transformations

Validations and transformations are core concerns when dealing with user input or data flowing through the controller, service & repository layer.

Goal : To ensure only good, safe, correct data enters the business logic and database.

Collectively, these layers are called CSR (Controller - Service - Repository)

or a broader type of n-tier. They can be called presentation layer. This is fundamental in backend engineering.

If not this, everything falls under one monolithic architecture. Think of this as a restaurant:

① Control layer (waiter): The waiter ~~will~~ take order, if it makes sense will pass it to kitchen.

② Service layer (Chef):

The chef reads the order, makes decision and co-ordinates cooking with the right staff below.

③ Repository layer: (pantry):

The chef requests ingredients from pantry, pantry keeps track of what is stock etc.

Example: A customer buys an item from an online store.

① Controller layer: (API endpoint):

→ Handle the HTTP request

(-) Receives a request JSON with cart details,

Payment info, shipping info.

(-) Performs basic validation (fields missing)

(-) Calls the Service layer.

② Service layer:

File: OrderService

Responsibility: Contains all business rules and workflow for order processing.

what it does: ① verifies inventory

② Validates user payments with

PaymentService.

- (3) Checks business constraints
- (4) Calculates total & discounts etc
- (5) Calls multiple repositories to update inventory and insert new order.
- (6) Triggers async jobs (confirmation email, shipping).
- (7) Returns failures/success or error (out of stock)

(3) Repository layer: (Data Persistence layer)

(.) File: OrderRepository, Inventory Repository.

(.) It interacts with Database

What it does?

→ Methods like createOrder, reserveInventory, findProductbyID etc

→ Handles all SQL, ORM logic, so the service doesn't know/care about DB details

→ Enables easy changes (switch from MySQL → DynamoDB only update this layer).

(A) If we had this in production:

(1) Each layer is a part of your backend microservice likely hosted as Docker

Containers. or Lambda functions.

② Controllers are exposed to the web via an API Gateway or Application load balancers.

③ Service & Repo layer share code in container, organized in modules/packages.

④ DB is a managed service (AWS RDS or).
Secure and scalable.

→ Why is this used in Production?

① Scalable: Can have hundreds of endpoints, workflows, and database tables, each clearly separated.

② Maintainable: Easy to change rules (Discounts, validation), add new feature or migrate databases.

③ Testable: Can unit test, order creation,

simulate errors without touching prod.

II Validations

It is the process of checking data to ensure it meets certain criteria before it is accepted, processed or stored. The main idea is to prevent bad, unsafe or corrupted data from entering the system.

(*) Why?

- Data Integrity: Ensure that only properly structured, logical and complete data gets saved and saved.
- Security: Prevent SQL injections, XSS or invalid calls.
- Business consistency: Enforce rules important to business logic, eg qty can't be negative.
- Error Prevention: Catch errors easily to avoid larger system failure downstream

Types of Validations

① Syntactic (Format) Validations

→ Checks the format (type, length, required).

② Semantic (logical) Validation.

→ Ensures data makes sense or not,

③ Cross-Relational Validation:

Checks relations b/w multiple fields or objects.

eg: does startdate occur after endDate?

④ Uniqueness & Existence:

Confirms that the data is unique or relates to something that exists.

eg: what if email already registered?

⑤ Custom/Domain-specific Validation:

Any special logic particular to application

⑥ Where do these happen?

(-) Frontend : For UX ,

(-) Controller : Initial checks -

(-) Service Layer : Business logic , cross entity check.

(-) Repository/DB : Integrity & final constraints

II Transformations

Transformation is the process of changing data from one format, structure or value to another to make it usable, consistent, or appropriate. It is crucial in software engineering.

Based on this, there are some types of transformations:

① Constructive : (a) Adding new fields

(b) Create new data from existing.

② Destructive : Remove unnecessary data .

③ Esthetic : Standardize date format , trim white space.

④ Structural : Change schema

⑤ Enrichment: Adding geo inf from address:

(*) Where is transformation done?

(i) Frontend / UI: Simple display action

(i) Backend / Service layer:

(1) Construct new data

(2) Standardize & validate new data

(3) Obfuscate personal data.

(4) Used for ETL pipelines etc.

Front End Validation

→ Used for UX, UI

(Server side)

Backend Validation.

→ Security &
Data Integrity.